

TCS Prime Interview Revision Notes

DBMS • Networking • Operating Systems • Cloud Computing • Web Development • React • Node.js

Purpose: Short, interview-ready explanations and comparisons for rapid TCS Prime revision.

1. DBMS

Database

A database is an organized collection of data that can be stored, accessed, managed, and updated efficiently.

Interview answer: A database is an organized collection of related data that can be efficiently stored, retrieved, and managed.

DBMS vs RDBMS

DBMS: Software used to create, store, retrieve, and manage data. **RDBMS:** A DBMS based on the relational model, storing data in tables and supporting relationships and constraints.

- Examples of RDBMS: MySQL, PostgreSQL, Oracle, SQL Server.
- RDBMS generally supports relational constraints and SQL.

Keys

- **Super key:** Any set of attributes that uniquely identifies a row.
- **Candidate key:** A minimal super key.
- **Primary key:** The candidate key chosen to uniquely identify rows.
- **Alternate key:** A candidate key not selected as the primary key.
- **Foreign key:** Attribute(s) referencing a key in another table, used to maintain referential integrity.
- **Unique key:** Constraint that enforces uniqueness; NULL handling can vary by DBMS.
- **Composite key:** A key made from multiple columns.
- A table has one primary key constraint, but it may contain multiple columns. A table can have multiple candidate keys.
- A primary key cannot contain NULL. A foreign key may contain NULL unless restricted by NOT NULL.
- A foreign key can reference a candidate/unique key when supported by the DBMS and constraints.

Normalization

Normalization organizes relational data to reduce redundancy and prevent anomalies.

- **1NF:** Atomic values; no repeating groups.
- **2NF:** In 1NF and no partial dependency of a non-key attribute on part of a composite candidate key.
- **3NF:** In 2NF and no problematic transitive dependency of non-key attributes on a key.
- **BCNF:** For every non-trivial functional dependency $X \rightarrow Y$, X must be a super key.
- **Functional dependency:** $X \rightarrow Y$ means the value of X determines the value of Y .
- **Partial dependency:** A non-key attribute depends on only part of a composite key.
- **Transitive dependency:** A non-key attribute depends on another non-key attribute through the key.
- **Anomalies:** insertion, deletion, and update anomalies can result from poor design.
- **Denormalization:** intentionally adding redundancy to improve read performance or simplify queries, at the cost of extra storage and more complex updates.

Interview answer: A table can be in 3NF but not BCNF. BCNF is stricter than 3NF.

NULL vs 0

NULL means missing, unknown, or not applicable data; **0** is an actual numeric value.

- Use IS NULL / IS NOT NULL rather than = NULL.
- Aggregate behavior with NULL should be understood; for example COUNT(column) ignores NULL values.

Index

An index is a data structure that speeds up data retrieval by allowing the database to locate rows more efficiently, at the cost of storage and additional work on writes.

Interview answer: An index improves read/query performance but can increase storage usage and slow INSERT, UPDATE, and DELETE operations.

View

A view is a virtual table defined by a query. It usually stores the query definition rather than a separate copy of the underlying data.

- Uses: security, abstraction, reusable complex queries.

Transactions & ACID

- **Transaction:** A logical unit of database work that is committed or rolled back as a whole.
- **Atomicity:** All or nothing.
- **Consistency:** Preserves database rules/constraints.
- **Isolation:** Concurrent transactions behave according to the chosen isolation level.
- **Durability:** Committed changes survive failures.

Schema vs Instance

Schema: Structure/design of the database. **Instance:** The actual data stored at a particular point in time.

Data Abstraction

- **Physical level:** how data is stored.
- **Logical level:** what data and relationships exist.
- **View level:** what individual users/applications see.

Relational Model

Represents data as relations (tables) consisting of rows/tuples and columns/attributes, with keys and constraints defining relationships and integrity.

Cardinality

Cardinality describes the number of entities/records that can participate in a relationship, such as 1:1, 1:M, and M:N.

Data Redundancy

Data redundancy means unnecessary duplication of the same data, which can waste storage and cause inconsistency.

Deadlock & Locking

A DBMS deadlock occurs when transactions wait indefinitely for resources held by each other.

- **Locking:** Controls concurrent access to shared data. Common concepts include shared/read locks and exclusive/write locks.
- Deadlock handling can involve prevention, avoidance, detection and recovery, or timeouts.

MongoDB

MongoDB is a document-oriented NoSQL database that stores data in BSON documents organized into collections.

- **ObjectId:** Common default MongoDB identifier type; designed to be unique and encodes timestamp-related information among other components.
- **Embedding:** Store related data inside a document; useful when data is usually accessed together.
- **Referencing:** Store a reference to another document; useful for separate or highly shared data.
- **Indexing:** Improves query performance but adds storage and write overhead.
- **Aggregation:** Pipeline-based processing of documents for filtering, grouping, transformation, and analysis.
- **\$lookup:** Aggregation stage used to combine documents from another collection, similar conceptually to a join.
- **Replication:** Maintaining copies of data across members for availability and redundancy.
- **Replica set:** A group of MongoDB instances maintaining the same data set, with an elected primary and secondary members.
- **Sharding:** Distributing data across multiple servers/shards to scale storage and throughput horizontally.
- **Transactions:** MongoDB supports multi-document transactions, including across collections, with appropriate deployment/configuration.

PostgreSQL vs MySQL

Both are popular relational DBMSs. PostgreSQL is known for strong standards support, advanced SQL features, extensibility, and complex workloads. MySQL is widely used, easy to deploy, and common in web applications. The right choice depends on workload, ecosystem, and requirements.

SQL & Clauses

- **SELECT:** retrieves data.
- **FROM:** specifies source tables.
- **WHERE:** filters rows before grouping.
- **GROUP BY:** groups rows for aggregation.
- **HAVING:** filters groups after aggregation.
- **ORDER BY:** sorts the result.
- **JOIN:** combines related rows from tables.
- **DISTINCT:** removes duplicate result rows.
- **LIMIT/OFFSET:** restricts and paginates results in supported SQL dialects.

Aggregate Functions

- COUNT(), SUM(), AVG(), MIN(), MAX().

Aggregate functions calculate a value over a set of rows, often with GROUP BY.

SQL Joins

- **INNER JOIN:** matching rows from both tables.
- **LEFT JOIN:** all rows from left table plus matching right rows.
- **RIGHT JOIN:** all rows from right table plus matching left rows.
- **FULL OUTER JOIN:** all rows from both sides, with NULLs where no match exists (support varies by DBMS).
- **CROSS JOIN:** Cartesian product.
- **SELF JOIN:** a table joined with itself.

ER Model

The Entity-Relationship model represents entities, attributes, and relationships before designing the relational schema.

2. Computer Networks

OSI Model

- 7 layers: Physical → Data Link → Network → Transport → Session → Presentation → Application.
- **Physical:** bits/signals. **Data Link:** frames/MAC. **Network:** IP/routing. **Transport:** TCP/UDP. **Session:** sessions. **Presentation:** format/encryption/compression concepts. **Application:** protocols such as HTTP, DNS, SMTP.

TCP/IP Model

Common 4-layer view: Link → Internet → Transport → Application.

- TCP is reliable and connection-oriented; UDP is connectionless with lower overhead and no built-in delivery guarantee.

TCP 3-Way Handshake

```
Client → SYN → Server
Client ← SYN-ACK ← Server
Client → ACK → Server
```

It establishes a TCP connection and synchronizes sequence information before data transfer.

IP Address

An IP address identifies a network interface/host at the network layer for routing packets.

- **IPv4:** 32-bit address, e.g. 192.168.1.10.
- **IPv6:** 128-bit address, designed to provide a much larger address space.

MAC vs IP

- **MAC:** link-layer hardware/interface address used for local network delivery.
- **IP:** logical network-layer address used for routing across networks.

Hub vs Switch vs Router

- **Hub:** broadcasts incoming data to all ports; largely obsolete.
- **Switch:** forwards frames based on MAC addresses within a LAN.
- **Router:** forwards packets between networks based on IP routing.

DNS

DNS translates domain names such as example.com into IP addresses and can provide other DNS records.

DHCP

DHCP automatically provides network configuration such as IP address, subnet mask, default gateway, and DNS server information to clients.

HTTP vs HTTPS

HTTP is the web application protocol. HTTPS is HTTP protected by TLS, providing encryption, integrity protection, and server authentication.

Port Number

A port identifies a logical endpoint for network services on a host. Common examples: HTTP 80, HTTPS 443, DNS 53, SSH 22.

TLS / SSL

TLS is the modern protocol used to secure communications such as HTTPS. SSL is an older predecessor and is deprecated/insecure. Say **TLS**, not 'SSL', when referring to modern HTTPS security.

HTTP Status Codes

- **1xx:** informational.

- **2xx:** success — 200 OK, 201 Created, 204 No Content.
- **3xx:** redirection — 301, 302, 304.
- **4xx:** client-side errors — 400, 401, 403, 404, 409, 429.
- **5xx:** server-side errors — 500, 502, 503, 504.

Cookies vs Sessions

- **Cookie:** small data stored by the browser and sent with applicable requests.
- **Session:** server-side or token-backed mechanism used to maintain user state across requests.
- A common session design stores a session identifier in a secure cookie while session data remains server-side.

Firewall

A firewall controls network traffic according to configured security rules, allowing or blocking traffic based on characteristics such as source, destination, port, protocol, and application context.

Proxy

A proxy is an intermediary that sends requests on behalf of a client or server. It can provide caching, filtering, access control, logging, or privacy features depending on its role.

VPN

A VPN creates an encrypted tunnel between endpoints over an untrusted network, commonly used for secure remote access or connecting networks.

NAT

Network Address Translation maps addresses/ports between private and public addressing contexts. It commonly allows private IPv4 hosts to access the Internet through a public address.

Latency, Bandwidth, Throughput

- **Latency:** time delay for data to travel/receive a response.
- **Bandwidth:** theoretical/maximum data-carrying capacity.
- **Throughput:** actual achieved data transfer rate.

3. Operating Systems

Operating System

An operating system manages hardware and provides services and abstractions for applications, including process, memory, file, device, and security management.

Thread

A thread is the smallest unit of execution scheduled within a process in common OS models. Threads in the same process share resources such as memory, while each thread has its own execution state and stack.

Context Switching

Context switching is the process of saving the state of one running process/thread and restoring another so the CPU can switch execution.

Schedulers

- **Long-term scheduler:** controls admission of jobs into the system; less common in modern general-purpose systems.
- **Short-term/CPU scheduler:** selects the next ready process/thread to run.
- **Medium-term scheduler:** may suspend/resume processes, often related to swapping in traditional models.

Deadlock

Deadlock is a state where processes/transactions wait indefinitely for resources held by one another.

- Four Coffman conditions: mutual exclusion, hold and wait, no preemption, circular wait.

Banker's Algorithm

Banker's Algorithm is a deadlock-avoidance algorithm that grants a resource request only if the resulting state remains safe.

- It uses maximum demand, allocation, available resources, and $\text{need} = \text{maximum} - \text{allocation}$.
- The key idea is to test whether there exists a safe sequence before granting resources.

Other Deadlock Approaches

- **Prevention:** design the system so at least one necessary deadlock condition cannot hold.
- **Avoidance:** make allocation decisions based on future resource needs; Banker's Algorithm is an example.
- **Detection and recovery:** allow deadlocks, detect them, then recover.
- **Timeouts:** practical mechanism in some systems, though not a general deadlock solution.

Semaphore

A semaphore is a synchronization primitive used to control access to shared resources.

- **Binary semaphore:** value commonly 0/1.
- **Counting semaphore:** value represents multiple available units of a resource.

Race Condition

A race condition occurs when the result depends on the timing/order of concurrent operations accessing shared state.

Critical Section

A critical section is the part of a program that accesses shared data/resources and therefore requires synchronization to prevent unsafe concurrent access.

- A good synchronization design aims for mutual exclusion and correct coordination.

Virtual Memory

Virtual memory gives processes the abstraction of a large private address space and allows parts of memory to be stored on secondary storage when necessary.

- Benefits: process isolation, larger logical address space, and flexible memory management.

Paging

Paging divides virtual memory into fixed-size pages and physical memory into fixed-size frames. Page tables map pages to frames.

Segmentation

Segmentation divides memory into variable-size logical segments such as code, data, and stack, with each segment having a base and limit in classic models.

Kernel

The kernel is the core part of an operating system that manages hardware resources and provides privileged services to applications.

- Responsibilities include process scheduling, memory management, device management, file systems, networking, and security mechanisms.

4. Cloud Computing & AWS

Cloud Computing

Interview answer: Cloud computing is the on-demand delivery of computing resources such as compute, storage, databases, and networking over the Internet, generally with usage-based pricing.

- Characteristics: on-demand self-service, resource pooling, broad network access, measured service, scalability, and elasticity.
- **IaaS:** infrastructure such as virtual machines and storage.
- **PaaS:** managed application platform.
- **SaaS:** complete software delivered as a service.
- Deployment models: public, private, hybrid, multi-cloud.

EC2

Amazon EC2 provides resizable compute capacity as virtual servers called instances.

EBS

Amazon EBS is persistent block storage primarily used with EC2. Think of it as a virtual disk.

- gp3: general-purpose SSD. io2: high-performance IOPS-oriented SSD. st1/sc1: HDD options for suitable throughput-oriented workloads.
- EBS snapshots provide point-in-time backups that can be used to create volumes.

S3 Bucket

An S3 bucket is a container for objects such as files, images, logs, backups, and static assets.

- **Object:** stored data. **Key:** identifier/name of the object.
- Important features: versioning, lifecycle rules, encryption, access policies, Block Public Access, replication, event notifications.
- S3 is object storage; EBS is block storage.

RDS

Amazon RDS is a managed relational database service supporting engines such as MySQL and PostgreSQL.

- AWS handles many infrastructure/administrative tasks such as provisioning, backups, patching, and maintenance.
- **Multi-AZ:** primarily for high availability and failover.
- **Read Replica:** primarily for read scaling.

Interview answer: RDS is a managed relational database service; I would place it in private subnets and protect it with security groups rather than exposing the database directly to the Internet.

Lambda

AWS Lambda is a serverless, event-driven compute service that runs code without requiring the developer to manage servers.

- Common triggers: S3, API Gateway, EventBridge, SQS, SNS, DynamoDB Streams.
- Lambda is good for event-driven and short-running workloads within service limits.
- EC2 gives more control; Lambda removes most server management.

Elastic Beanstalk

Elastic Beanstalk is a PaaS service for deploying and managing web applications while AWS handles much of the underlying environment setup.

- It can provision/manage resources such as EC2, load balancing, and Auto Scaling.

- It is **not serverless**; applications run on managed infrastructure such as EC2.

API Gateway

Amazon API Gateway is a managed service for creating, publishing, securing, monitoring, and managing APIs.

- Can integrate with Lambda and backend services.
- Features include authentication/authorization integrations, throttling, stages, monitoring, and API management.

Memory: API Gateway manages APIs; Load Balancer distributes traffic; Lambda executes serverless functions.

Auto Scaling

Auto Scaling automatically adjusts compute capacity based on workload demand. EC2 Auto Scaling Groups define minimum, desired, and maximum capacity.

- **Scale out:** add instances. **Scale in:** remove instances.
- Policies can be dynamic, scheduled, or predictive depending on the service/setup.
- It improves scalability, availability, fault recovery, and cost efficiency.

Load Balancer

A load balancer distributes incoming traffic across multiple healthy targets.

- **ALB:** Layer 7, HTTP/HTTPS, host/path-based routing.
- **NLB:** Layer 4, TCP/UDP/TLS, high performance and low latency.
- **GWLB:** deploys/scales network/security appliances.
- Load Balancer distributes traffic; Auto Scaling changes the number of instances.

Other AWS Services to Know

- **VPC:** logically isolated virtual network in AWS.
- **IAM:** manages identities and permissions; follow least privilege.
- **CloudWatch:** monitoring, metrics, logs, alarms.
- **Route 53:** DNS and domain routing service.
- **Security Group:** stateful virtual firewall for resources such as EC2.
- **NAT Gateway:** lets resources in private subnets initiate outbound Internet connections without allowing unsolicited inbound Internet connections.
- **High availability:** design to keep services available despite component/AZ failures.
- **Fault tolerance:** ability to continue operating despite failures, often using redundancy.
- **Scalability:** ability to handle increased workload by adding capacity.

Three-Tier Architecture

```

Users
↓
Route 53
↓
Load Balancer
↓
Presentation / Web Tier
↓
Application / API Tier
↓
Database Tier (RDS)

```

A common cloud architecture separates presentation, application logic, and data storage into tiers. Use private networking, security groups, Auto Scaling, and Multi-AZ design as appropriate.

10× User Scalability Question ■

Interview answer: If users increased by 10×, I would first identify bottlenecks and design for horizontal scaling. Route 53 can route users to the application endpoint, a load balancer can distribute traffic, and an Auto Scaling Group can add EC2 instances across multiple Availability Zones. I would use CloudWatch for monitoring and scaling signals, caching where appropriate, database read replicas or other database scaling strategies for read-heavy workloads, and S3 for static assets. I would also review VPC capacity, security, rate limits, and application/database performance through load testing.

Redis / Distributed System

Redis is an in-memory data store commonly used for caching, session storage, counters, queues, and other low-latency workloads. In distributed systems, caching can reduce database load and improve response time.

- For a production design, consider cache invalidation, TTLs, consistency, high availability, and failure behavior.

5. Web Development

HTML

HTML structures web content.

- Know: headings, paragraphs, links, images, forms, tables, and semantic tags such as header, nav, main, section, article, footer.

CSS

CSS controls styling and layout.

- Know: selectors, box model, margin, padding, border, Flexbox, Grid, positioning, responsive design, media queries.

JavaScript

JavaScript is a programming language used for web interactivity and application logic; it can also run server-side with Node.js.

- Know: let/const/var, data types, functions, arrays, objects, loops, promises, async/await, DOM, events, JSON, Fetch API.

HTTP / HTTPS

HTTP enables client-server web communication. HTTPS is HTTP secured with TLS.

REST API

REST is an architectural style commonly used for web APIs. Resources are exposed through URLs and manipulated using HTTP methods.

- GET: retrieve; POST: create; PUT: replace/update; PATCH: partial update; DELETE: remove.
- REST APIs are typically stateless.

Frontend vs Backend

- **Frontend:** UI and client-side interaction — HTML, CSS, JavaScript, React.
- **Backend:** server-side logic, APIs, authentication, authorization, and database access — Node.js, Express, etc.

```
React + Vite → Fetch API → Node.js + Express → MongoDB
```

Authentication vs Authorization

- **Authentication:** verifies identity — 'Who are you?'
- **Authorization:** verifies permissions — 'What are you allowed to do?'
- Example: logging in is authentication; permission to delete users is authorization.

Input Validation

Input validation checks whether user input meets expected rules before processing.

- Always validate on the server; client-side validation improves UX but can be bypassed.
- Use type, length, format, range, and business-rule checks as appropriate.

Basic Web Security

- **HTTPS/TLS:** protects data in transit.
- **SQL Injection:** use parameterized queries/prepared statements.
- **XSS:** prevent unsafe script injection using output encoding, safe rendering, validation, and appropriate CSP.
- **CSRF:** use defenses such as CSRF tokens and SameSite cookies where applicable.
- **Passwords:** use strong password hashing; never store plaintext passwords.
- **Authorization:** enforce permissions server-side.

- **Rate limiting:** reduce abuse and excessive requests.
- **Cookies:** use Secure, HttpOnly, and appropriate SameSite settings when applicable.

6. React

What is React?

Interview answer: React is a JavaScript library developed by Meta for building component-based user interfaces.

- React uses reusable components, props, state, hooks, JSX, and efficient UI updates.
- In your Multi-Cloud Cost Dashboard: React + Vite frontend → Fetch API → Node.js/Express backend → MongoDB.

Components

Reusable, independent UI pieces that can receive props and manage state.

JSX

A syntax extension that allows HTML-like UI syntax inside JavaScript.

Props

Inputs passed from parent to child components. They are read-only from the receiving component's perspective.

State

Component-managed data that can change over time and trigger UI updates.

Props vs State

- Props: passed into a component; state: managed by the component.

useState

```
const [count, setCount] = useState(0);
```

useState lets a functional component manage state.

useEffect

useEffect handles side effects such as fetching data, subscriptions, and synchronization with external systems.

```
useEffect(() => {  
  fetch('/api/costs')  
    .then(res => res.json())  
    .then(data => setCosts(data));  
}, []);
```

Virtual DOM

React maintains a virtual representation of the UI and determines necessary updates to the browser DOM.

Lists & Keys

When rendering lists, provide stable keys so React can identify items across renders.

Conditional Rendering

```
{loading ? Loading... : }
```

React vs JavaScript

JavaScript is a programming language; React is a JavaScript library for UI development.

React vs Angular

React is a library with a flexible ecosystem; Angular is a more opinionated full framework. Both support component-based development.

7. Node.js

What is Node.js?

Interview answer: Node.js is a JavaScript runtime environment built on Chrome's V8 engine that allows JavaScript to run outside the browser, commonly on servers.

Node.js vs JavaScript

JavaScript is the language; Node.js is a runtime that executes JavaScript outside the browser.

Node.js vs Express.js

Node.js provides the runtime; Express.js is a web framework running on Node.js that simplifies HTTP servers and REST APIs.

Event-Driven & Non-Blocking I/O

Node.js commonly uses an event-driven, asynchronous, non-blocking I/O model, which is well suited to I/O-intensive applications.

Event Loop

The event loop coordinates asynchronous operations and callbacks/promises while JavaScript execution uses the main thread in the standard Node.js model.

npm

npm is commonly used to install and manage Node.js/JavaScript packages.

```
npm install express
```

Node + Express Example

```
const express = require('express');
const app = express();

app.get('/api/users', (req, res) => {
  res.json({ message: 'Users fetched' });
});

app.listen(5000);
```

Node + MongoDB

```
React + Vite
↓
Fetch API
↓
Node.js + Express
↓
MongoDB Atlas
```

Interview answer: I used Node.js as the server-side JavaScript runtime and Express.js to build REST API endpoints. The React frontend sends HTTP requests to the Express backend, which performs CRUD operations on MongoDB.

Node.js Limitations

CPU-heavy synchronous work can block the main JavaScript execution thread. For CPU-intensive workloads, consider worker threads, multiple processes, background jobs, or separate compute services.

8. Rapid-Fire Final Revision

Topic	One-line answer
Cloud	On-demand compute, storage, databases, networking.
EC2	Virtual server / compute.

Topic	One-line answer
EBS	Persistent block storage / virtual disk.
S3	Object storage using buckets and objects.
RDS	Managed relational database.
Lambda	Serverless, event-driven compute.
Elastic Beanstalk	PaaS for deploying/managing applications; not serverless.
API Gateway	Create/manage/secure APIs.
Auto Scaling	Automatically adjusts compute capacity.
Load Balancer	Distributes traffic across healthy targets.
VPC	Logically isolated AWS network.
IAM	Identity and access management.
CloudWatch	Monitoring, metrics, logs, alarms.
Route 53	DNS and routing.
NAT Gateway	Outbound Internet access for private subnet resources.
Authentication	Who are you?
Authorization	What can you do?
REST	Resource-oriented API style using HTTP methods.
React	Component-based JavaScript UI library.
Node.js	JavaScript runtime outside the browser.
Express.js	Node.js web framework.
MongoDB	Document-oriented NoSQL database.
ACID	Atomicity, Consistency, Isolation, Durability.
Deadlock	Processes/transactions wait indefinitely for each other's resources.
Semaphore	Synchronization primitive for shared resources.
Virtual Memory	Abstraction of a large process address space backed by physical memory/storage.

Final Interview Tip

For each technical topic, answer in this order: **definition** → **purpose** → **simple example** → **comparison/use case**. Keep the first answer to 20–40 seconds, then expand only if the interviewer asks follow-up questions.